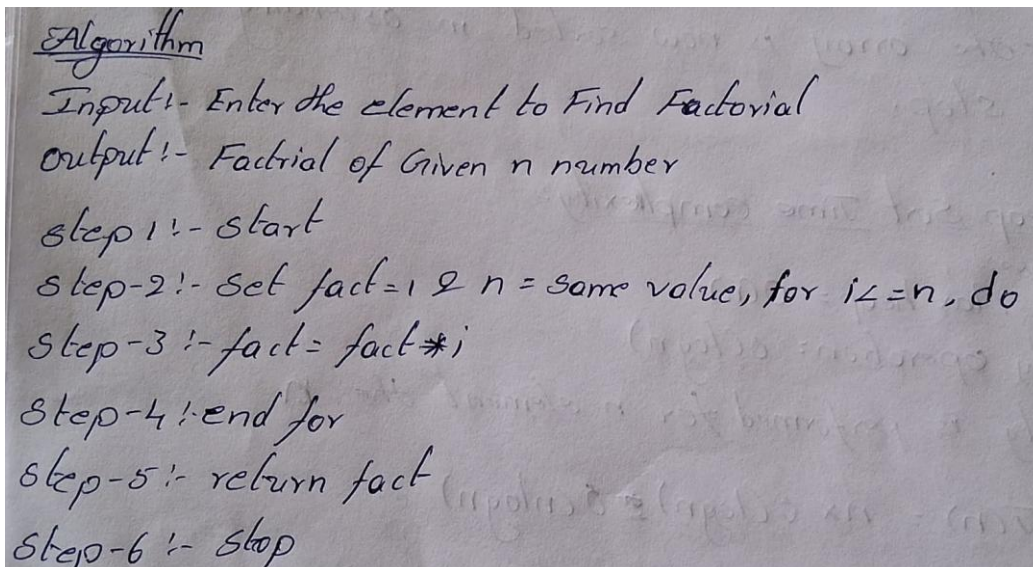


Practical: 4	Implementation and Time analysis of factorial program using iterative and recursive method
12/08/2026	

Aim: Implementation and Time analysis of factorial program using iterative and recursive method.

Factorial program using iterative

Algorithm:



Algorithm
Input:- Enter the element to Find Factorial
Output:- Factorial of Given n number
Step 1:- start
Step 2:- set fact=1 & n = some value, for i=1 to n, do
Step 3:- fact = fact * i
Step 4:- end for
Step 5:- return fact
Step 6:- stop

Program:

```
#include <iostream>
using namespace std;
class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;
        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }
        return result;
    }
};
int main()
{
    int n,result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.fact(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output:

```
Output

Enter the value to find its factorial: 5
Factorial of 5 is 120

=== Code Execution Successful ===
```

Time Complexity:

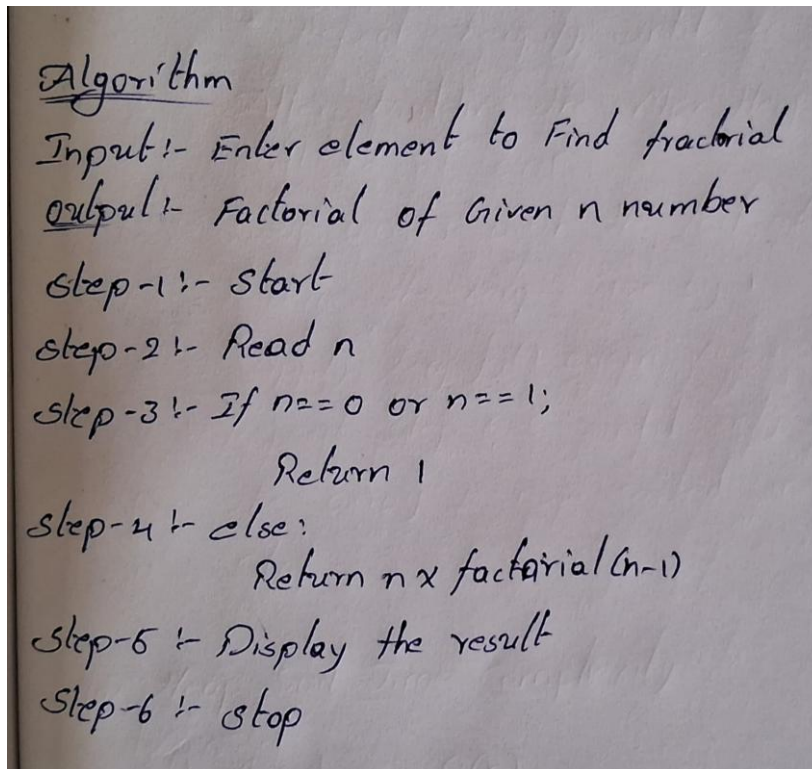
Best case
For loop doesn't execute
Number of iteration = 0
 $T(n) = O(1)$

worst case
Number of iterations = n
 $T(n) = C_1 + C_2 n + C_3 n$
 $= C_1 + (C_2 + C_3) n$
 $= O(n)$

Average Case:-
Number of iteration = n
 $T(n) = C_1 + C_2 n$
 $= (C_1 + C_2) n$
 $= O(n)$

Factorial program using recursive method

Algorithm:



Algorithm
Input:- Enter element to Find factorial
Output:- Factorial of Given n number
Step-1:- Start
Step-2:- Read n
Step-3:- If $n=0$ or $n=1$;
 Return 1
Step-4:- else:
 Return $n \times \text{factorial}(n-1)$
Step-5:- Display the result
Step-6:- stop

Program code:

```
#include <iostream>
using namespace std;
class Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
        else
        {
            int result = n * factr(n - 1);
            return result;
        }
    }
};

int main()
{
    int n, result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.factr(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output:

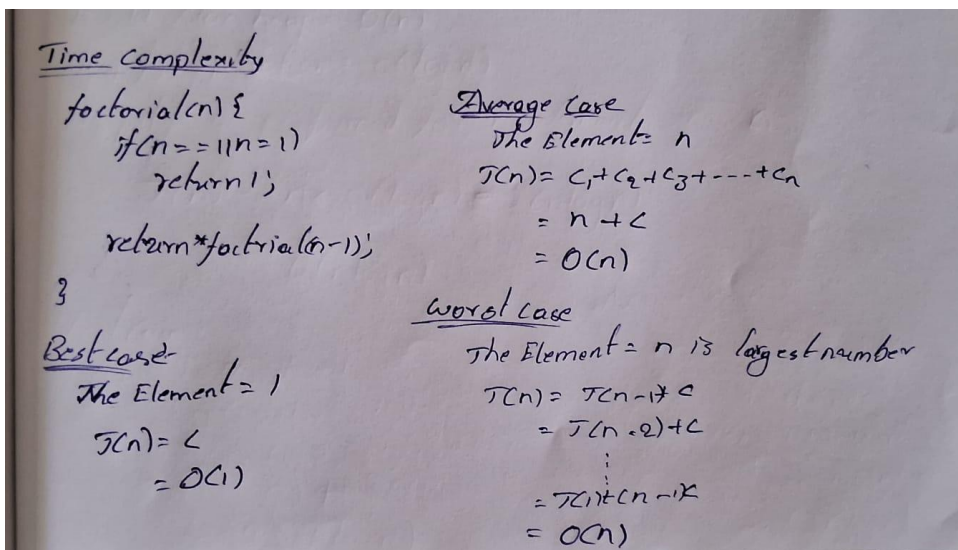
```
Output

Enter the value to find its factorial: 4
Factorial of 4 is 24

=== Code Execution Successful ===
```

+

Time complexity:



Time complexity

factorial(n) {
 if (n == 1 || n == 0)
 return 1;
 return n * factorial(n-1);
}

Best case
The Element = 1
 $T(n) = C$
 $= O(1)$

Average case
The Element = n
 $T(n) = C_1 + C_2 + C_3 + \dots + C_n$
 $= n + C$
 $= O(n)$

Worst case
The Element = n is largest number
 $T(n) = T(n-1) + C$
 $= T(n-2) + C$
 $\vdots$
 $= T(1) + (n-1)C$
 $= O(n)$

Conclusion

The factorial program can be implemented using both iterative and recursive methods. The iterative method uses a loop to calculate the factorial, while the recursive method calls the same function repeatedly until it reaches the base condition. Both methods produce the same result, but the iterative method generally uses less memory, whereas the recursive method is simpler and demonstrates the concept of recursion.